pact-jvmではじめる コンシューマー駆動契約

2017/4/24 JJUG ナイト・セミナー「テスティング特集」

大中浩行(@setoazusa)

この作品は クリエイティブ・コモンズ 表示 4.0 国際 ライセンスの下に提供されています。

Copyright 2017 Hiroyuki Onaka

最近よく聞く話

- ユニットテスト
  - 「このコンポーネントのテストモックだらけでわかりづらいんですけど…」
- エンドツーエンドのテスト
  - 「テストケースが増えてきてメンテナンスがつらいです…」

Copyright 2017 Hiroyuki Onaka

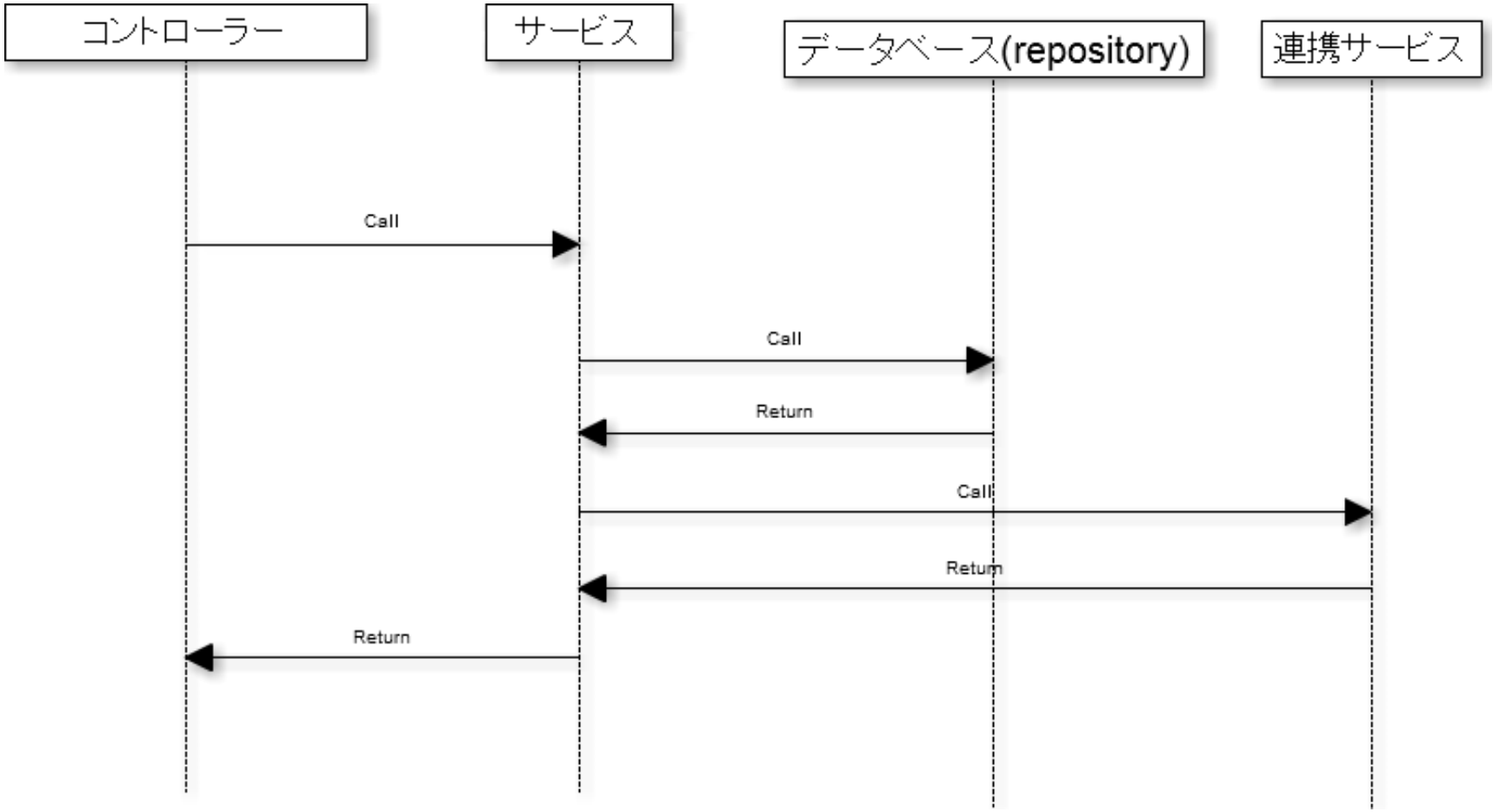
今日は、そんな自動テストに悩むみなさんへの処方箋として、「コンシューマー駆動契約」についてお話しします。

Copyright 2017 Hiroyuki Onaka

問題点の整理

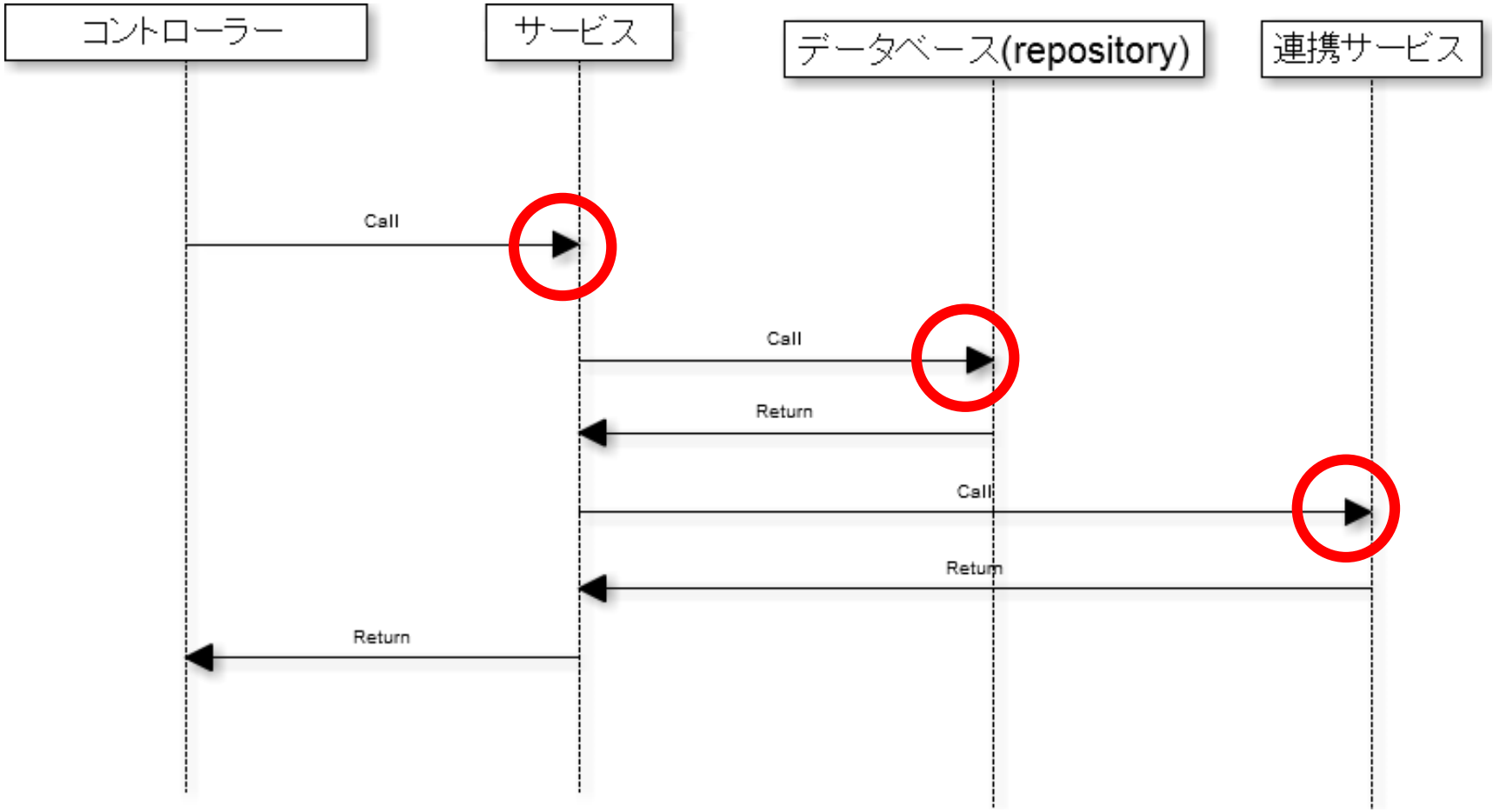
Copyright 2017 Hiroyuki Onaka

モックか直接呼び出すか、それが問題



Copyright 2017 Hiroyuki Onaka

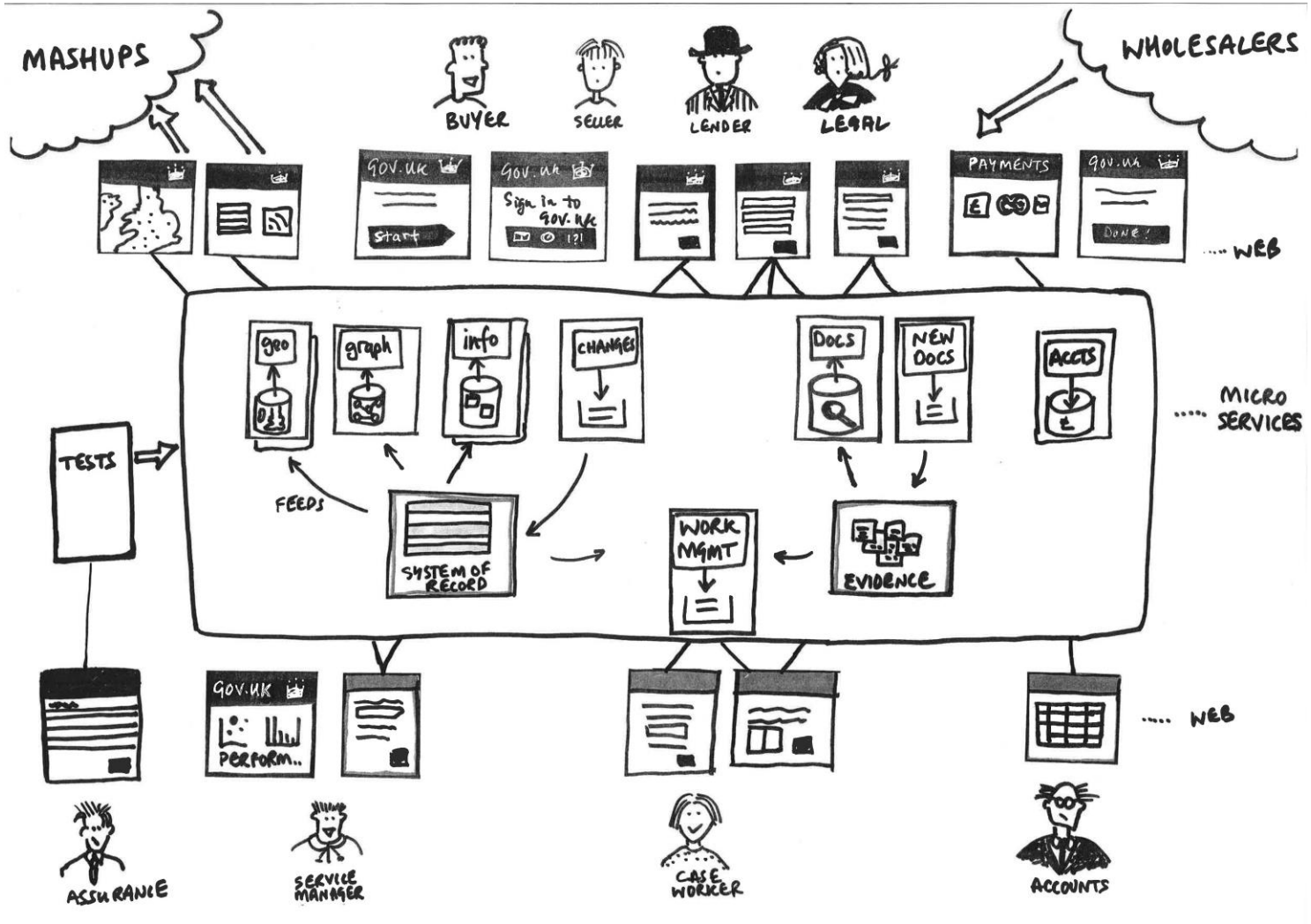
今までなら、モックを使うかどうかの判断は簡単だった



Copyright 2017 Hiroyuki Onaka

By Paul Downey (CC-BY 2.0)
https://www.flickr.com/photos/psd/13109673843

「マイクロサービスアーキテクチャー」



Copyright 2017 Hiroyuki Onaka

BY Rose and Trev Clough(CC-BY SA 2.0) http://www.geograph.org.uk/photo/1490900

複雑になるサービスの依存関係

By Matthew Black from London, UK (325000 Uploaded by oxyman)
[CC BY-SA 2.0 (http://creativecommons.org/licenses/by-sa/2.0)], via Wikimedia Commons
https://commons.wikimedia.org/wiki/File%3AMock_up_of_a_British_Rail_Class_325_cab_at_the_National_Railway_Museum.jpg

「なんかモックだらけになるんですけど…」

よろしい、ならばエンドツーエンドだ

gulp

<1> gulp

```
bower jquery#1.9.1 - 3          cached https://github.com/jquery/jquery-dist.git#3.1.1
bower jquery#1.9.1 - 3       validate 3.1.1 against https://github.com/jquery/jquery-dist.git#1.9.1 - 3
bower jquery#1.9.1 - 3
bower jquery#1.9.1 - 3
bower jquery#1.9.1 - 3
bower jquery#1.9.1 - 3
bower jquery#1.9.1 - 3
bower jquery#1.9.1 - 3
bower bootstrap#~3.3.5
bower jquery#1.9.1 - 3

bootstrap#3.3.7 bower_comp
└── jquery#3.2.1

jquery#3.2.1 bower_compone
bower chai#*
bower chai#*
bower mocha#*
bower mocha#*
bower chai#^3.5.0
bower mocha#^3.2.0

chai#3.5.0 bower_component

mocha#3.2.0 bower_componen

azusa@DESKTOP-KHQL29U C:\U
> gulp serve
[21:55:18] Using gulpfile
[21:55:18] Starting 'serve
[21:55:18] Starting 'clear
[21:55:18] Starting 'wiredc
[21:55:18] Finished 'wiredc
[21:55:18] Finished 'serve
[21:55:18] Finished 'clear
[21:55:18] Starting 'style
[21:55:18] Starting 'scrip
[21:55:19] Starting 'fonts
[21:55:19] Finished 'style
[21:55:19] Finished 'scrip
[21:55:19] Finished 'fonts
[BS] Access URLs:
 ----------------------
```

webapp example

localhost:9000

webapp example

Home About Contact

'Allo, 'Allo!

Always a pleasure scaffolding your apps.

Splendid!

HTML5 Boilerplate

HTML5 Boilerplate is a professional front-end template for building fast, robust, and adaptable web apps or sites.

Bootstrap

エンドツーエンドテストつらい…

- データのセットアップつらい…
- 実行時間つらい…
- コード何もいじってないのにテスト落ちた、つらい…
- 昔あった「結合一発勝負」とどう違うんや

Copyright 2017 Hiroyuki Onaka

- 各種パブリッククラウド上でのサービス構築に代表されるように、テストを書くときに依存するサービス/ミドルウェアが増えた。
- マイクロサービスアーキテクチャーがその流れを加速している。

Copyright 2017 Hiroyuki Onaka

それにもかかわらず、テストをはじめとして開発の方法論がいわゆる三層アーキテクチャーに最適化されすぎているが故の問題。

Copyright 2017 Hiroyuki Onaka

ではどうするか

Copyright 2017 Hiroyuki Onaka

Sam Newman(著) 佐藤直生(監訳)
「マイクロサービスアーキテクチャ」

O'REILLY
オライリー・ジャパン

マイクロサービスアーキテクチャ

「ストーリーではなくジャーニーをテストする」

「これに対抗する最善の方法は、**少数**の中核となるジャーニーに焦点を絞ってシステム全体をテストする方法です。この中核となるジャーニーで対象になっていない機能は、互いに分離してサービスを分析するテストで対処する必要があります。このジャーニーは相互に合意され、共同で所有される必要があります。音楽専門店の例では、CD の注文、商品の返品、新規顧客の作成といった（高価値な対話であり極めて少数の）動作に焦点を絞るでしょう。」

Copyright 2017 Hiroyuki Onaka

「ストーリーではなくジャーニーをテストする」

どうやって…?

Copyright 2017 Hiroyuki Onaka

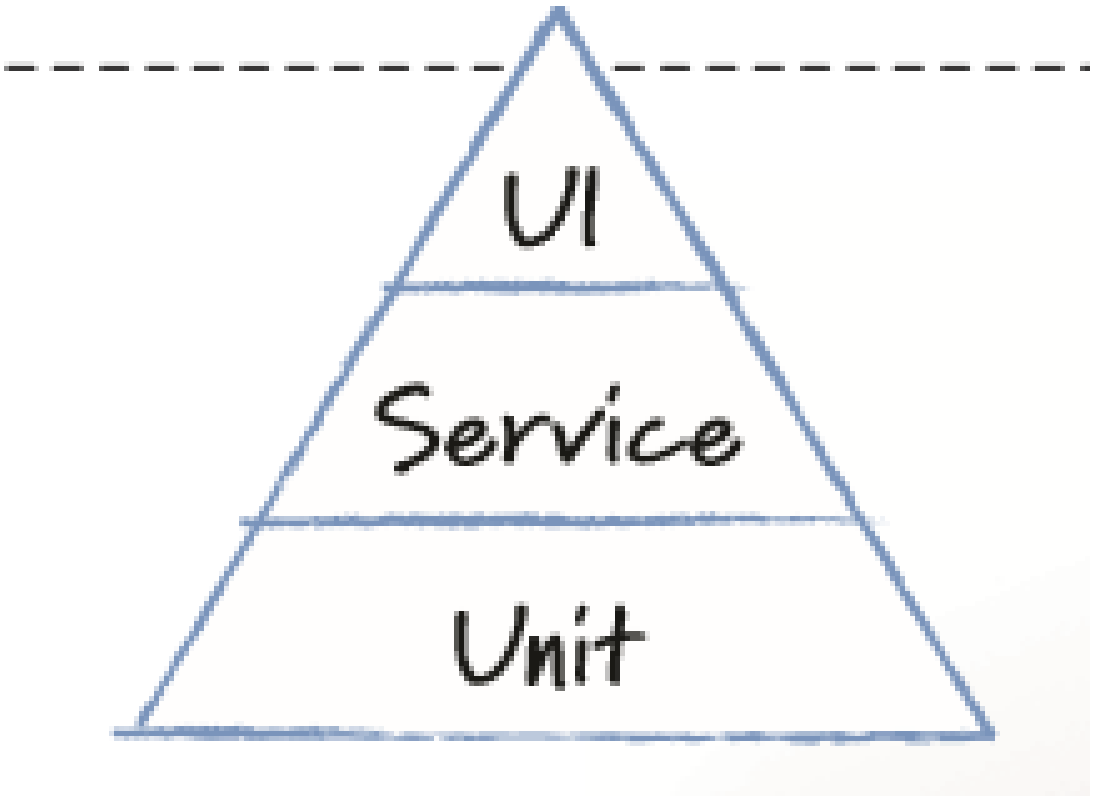
The Addison-Wesley Signature Series

A MIKE COHN SIGNATURE BOOK

Succeeding with Agile

Software Development Using Scrum

「テスト自動化ピラミッド」



Copyright 2017 Hiroyuki Onaka

「テスト自動化ピラミッド」

- テスト自動化のレイヤーを「UI」「Service」「Unit」の3つに分類したもの。
- 「Service」に対するテストが、アプリケーションのインターフェスに対するテストを、UI(ユーザーインターフェース)を迂回して実行することが特徴。

Copyright 2017 Hiroyuki Onaka

マイクロサービスアーキテクチャーにおけるサービステスト

- 外部に対するサービス呼び出しをモック化した上で、個々のサービスの機能に対して、ユーザーインターフェースを迂回して実行する。
- ユニットテストよりも広い範囲をカバーするテストを、エンドツーエンドのテストよりも安定かつ高速に実行できる。

Copyright 2017 Hiroyuki Onaka

- それでは、サービステストを実現する手段としてのコンシューマー駆動契約とその実装であるpact-jvmについて見てみましょう。

Copyright 2017 Hiroyuki Onaka

コンシューマー駆動契約とPact

Copyright 2017 Hiroyuki Onaka

コンシューマー駆動契約

- クライアント(コンシューマー)がサービス(プロバイダー)に対してどのような振る舞いを期待するかを記述したファイル(エクスペクテーション)を作成する。
- エクスペクテーションは、クライアントのビルド上で作成される。

Copyright 2017 Hiroyuki Onaka

- サービスは、クライアントからエクスペクテーションを受け取り、自らがエクスペクテーションの通り振る舞うかを、サービスのビルドの中で検証する。
- エクスペクテーションを作成する過程と検証する過程が、分割されていることが特徴。

Copyright 2017 Hiroyuki Onaka

Pact

- オーストラリアのオンライン不動産サービスRealEstate.com.auで開発された、コンシューマー駆動契約を使用したテストツールがオープンソース化されたもの。
- https://docs.pact.io/

Copyright 2017 Hiroyuki Onaka

- 当初はRubyで開発されていたが、その後他の言語へも移植。
- エクスペクテーションがJSONで記述されていて、特定の言語からは独立したフォーマットになっていることが特徴。

Copyright 2017 Hiroyuki Onaka

pact-jvm

- PactのJVM言語向け移植版
- https://github.com/DiUS/pact-jvm

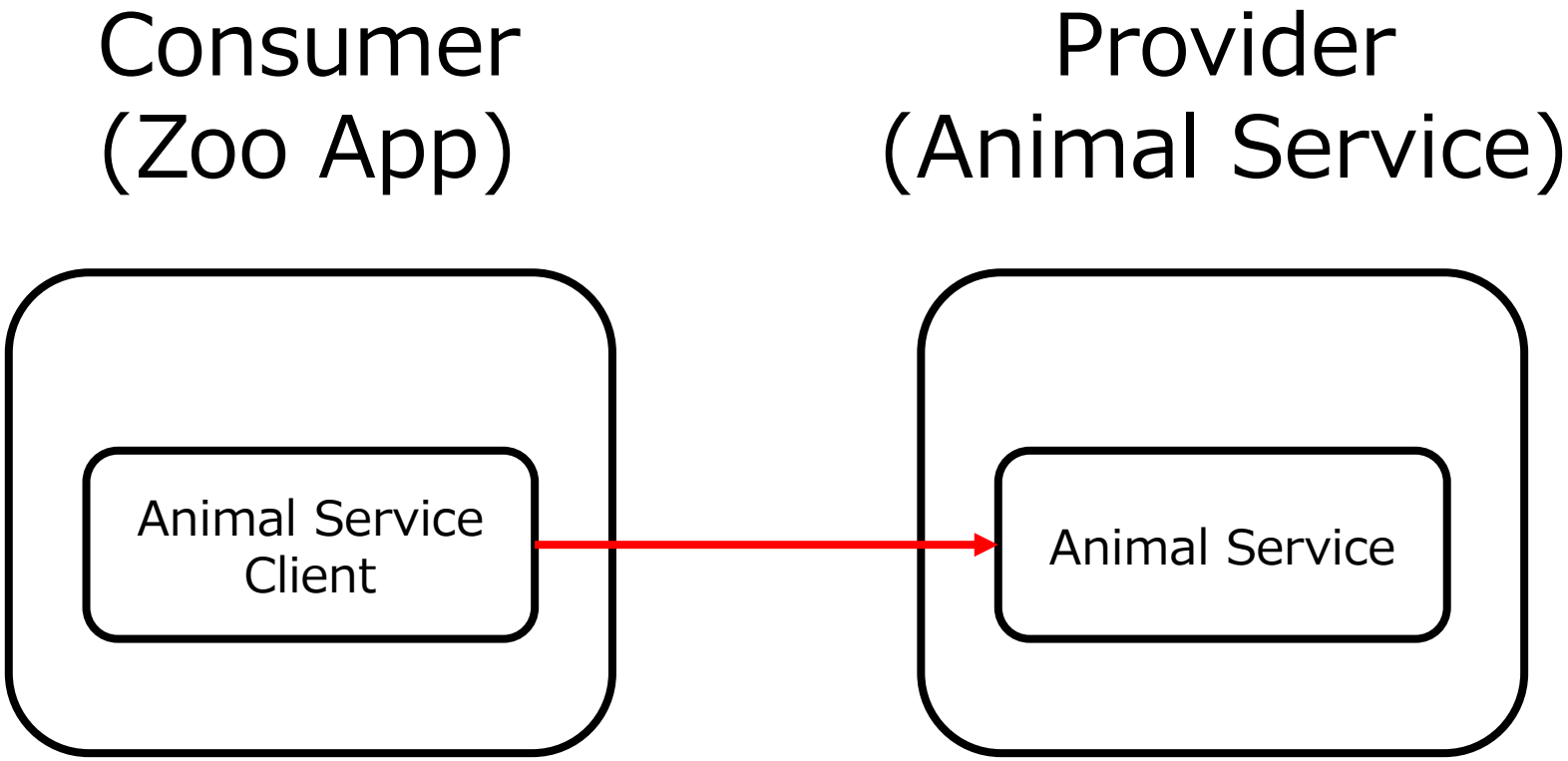
Copyright 2017 Hiroyuki Onaka

サンプルの解説

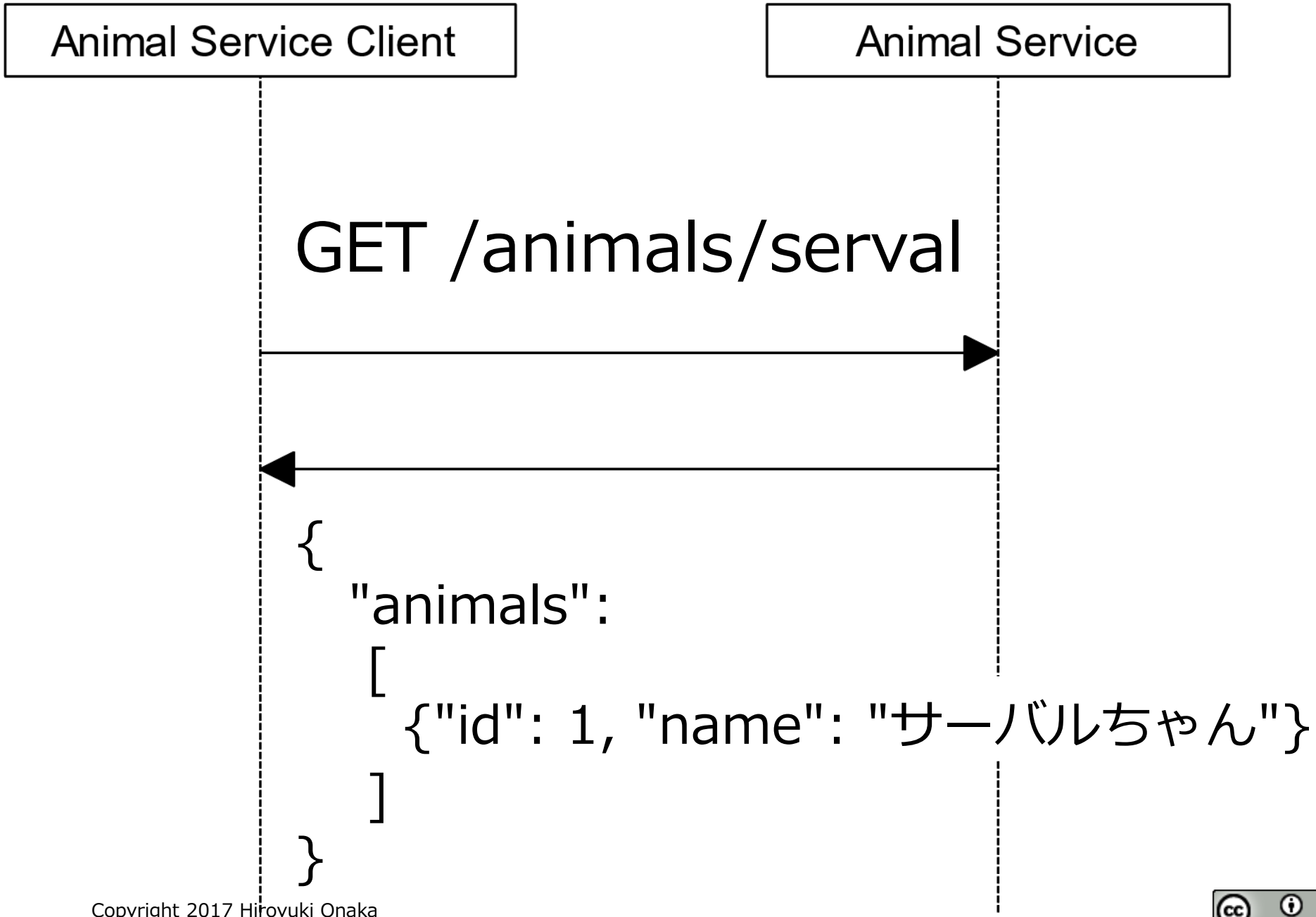
Copyright 2017 Hiroyuki Onaka

- 動物園で飼育されている動物のモデルを返すサービス
- そのサービスを使用して動物園の情報を扱うクライアント
- サンプルコードは以下にあります https://github.com/azusa/pact-jvm-example

Copyright 2017 Hiroyuki Onaka

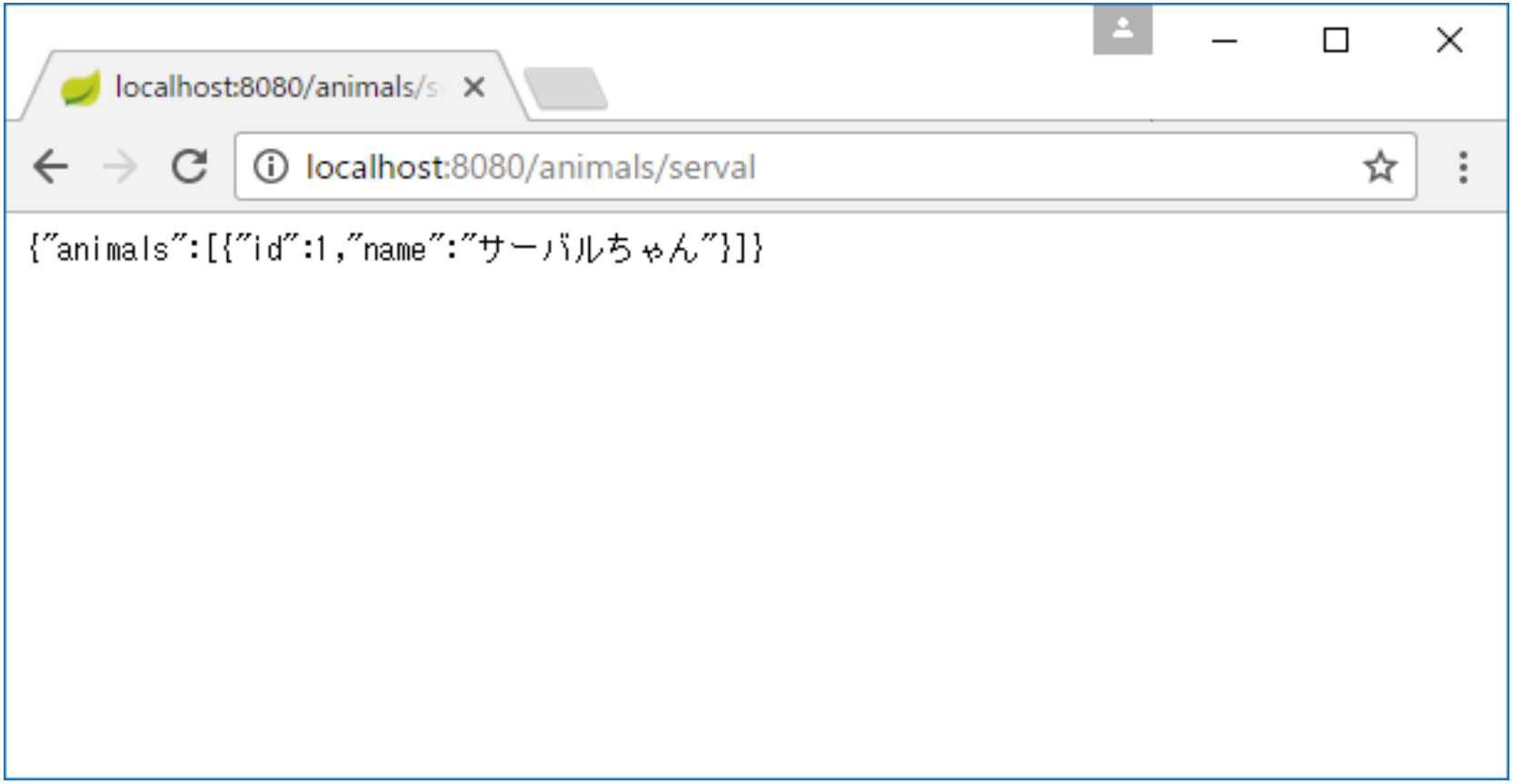


Copyright 2017 Hiroyuki Onaka



Copyright 2017 Hiroyuki Onaka

プロバイダーとしてのAnimal Service



Copyright 2017 Hiroyuki Onaka

```java
public class ProviderCarBookingRestClientTest {

    Map<String, String> headers = Collections.singletonMap("Content-Type", "application/json");

    @Rule
    public PactProviderRule provider = new PactProviderRule( provider: "test_provider",  host: "localhost",  port: 1234,  target: this);

    @Pact(provider="test_provider", consumer="test_consumer")
    public PactFragment configurationFragment(PactDslWithProvider builder) {
        return builder
            .given( state: "a serval exists")
            .uponReceiving( description: "a request for an serval")
            .path("/animals/serval")
            .method("GET")
            .willRespondWith()
            .headers(headers)
            .status(200)
            .body(
                    "{¥n" +
                            "¥"animals¥": [¥n" +
                            "    {¥n" +
                            "                    ¥"id¥": 1,¥n" +
                            "                    ¥"name¥": ¥"サーバルちゃん¥"¥n" +
                            "    }¥n" +
                            "]¥n" +
                            "}"
            )
            .toFragment();
    }

    @PactVerification("test_provider")
    @Test
    public void get_serval() throws IOException {
        ProviderCarBookingRestClient providerRestClient = new ProviderCarBookingRestClient();
        Animal response = providerRestClient.placeOrder( baseUrl: "http://localhost:1234",  name: "serval");
    }
}
```

Copyright 2017 Hiroyuki Onaka

```java
@Pact(provider="test_provider", consumer="test_consumer")
public PactFragment configurationFragment(PactDslWithProvider builder) {
    return builder
        .given( state: "a serval exists")
        .uponReceiving( description: "a request for an serval")
        .path("/animals/serval")
        .method("GET")
        .willRespondWith()
        .headers(headers)
        .status(200)
        .body(
                "{¥n" +
                        "¥"animals¥": [¥n" +
                        "    {¥n" +
                        "                    ¥"id¥": 1,¥n" +
                        "                    ¥"name¥": ¥"サーバルちゃん¥"¥n" +
                        "    }¥n" +
                        "]¥n" +
                        "}"
        )
        .toFragment();
}
```

Copyright 2017 Hiroyuki Onaka

エクスペくてーションの出力

クライアントでテストを実行すると、

target¥pacts¥test_consumer-test_provider.json に、JSONが出力されます。

Copyright 2017 Hiroyuki Onaka

Run: ProviderCarBookingRestClientTest (1) | AnimalContractTest | AnimalContractTest | AnimalsRepositoryTest

1 test passed – 2s 784ms

ProviderCarBookingRestClientTest (jp.fieldnotes.jjug.pact.provider.pact.consumer) 2s 784ms

get_serval 2s 784ms

```
"C:¥Program Files¥Java¥jdk1.8.0¥bin¥java" ...
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.

Process finished with exit code 0
```

Copyright 2017 Hiroyuki Onaka

```
{
    "provider": {
        "name": "test_provider"
    },
    "consumer": {
        "name": "test_consumer"
    },
    "interactions": [
        {
            "description": "a request for an serval",
            "request": {
                "method": "GET",
                "path": "/animals/serval"
            },
            "response": {
                "status": 200,
                "headers": {
                    "Content-Type": "application/json"
                },
                "body": {
                    "animals": [
```

```
            "response": {
                "status": 200,
                "headers": {
                    "Content-Type": "application/json"
                },
                "body": {
                    "animals": [
                        {
                            "id": 1,
                            "name": "\u30b5\u30fc\u30d0\u30eb"
                        }
                    ]
                }
            },
            "providerState": "a serval exists"
        }
    ],
    "metadata": {
        "pact-specification": {
            "version": "2.0.0"
```

サービス側のテスト

エクスペクテーション(先ほどのJSON)をコンシュマー(クライアント)から受け取って、プロバイダー(サービス)のビルドで、サービスの振る舞いを検証します。

Copyright 2017 Hiroyuki Onaka

```java
@RunWith(PactRunner.class)
@Provider("test_provider")
@PactFolder("../pact-jvm-example-consumer/target/pacts")
public class AnimalContractTest {

    private static  ExecutorService executor = Executors.newSingleThreadExecutor();

    @TestTarget
    public final Target target = new HttpTarget( port: 8080);

    @BeforeClass
    public static  void beforeClass() throws IOException, InterruptedException {
        executor.submit(
                () -> SpringApplication.run(PactJvmExampleProviderApplication.class, new String[]{}
        for (int i=0; i<300; i++){
            try (Socket s = new Socket( host: "localhost",  port: 8080)){
                    return;
            } catch (IOException e) {
            }
            Thread.sleep( millis: 100);
        }
        throw new IllegalStateException();
    }

    @State("a serval exists")
    public void stateChange() {}
```

```
@RunWith(PactRunner.class)
@Provider("test_provider")
@PactFolder("../pact-jvm-example-consumer/target/pacts")
public class AnimalContractTest {

    private static  ExecutorService executor = Executors.newSingleThreadExecutor();

    @TestTarget
    public final Target target = new HttpTarget( port: 8080);

    @BeforeClass
    public static  void beforeClass() throws IOException, InterruptedException {
        executor.submit(
                () -> SpringApplication.run(PactJvmExampleProviderApplication.class, new String[]{}
        for (int i=0; i<300; i++){
            try (Socket s = new Socket( host: "localho
                    return;
            } catch (IOException e) {
            }
            Thread.sleep( millis: 100);
        }
        throw new IllegalStateException();
    }

    @State("a serval exists")
    public void stateChange() {}
```

テスト対象のポートの指定

アプリケーションの起動

エクスペクテーションとのマッピング

実行結果

Run: ProviderCarBookingRestClientTest (1) | AnimalContractTest | AnimalContractTest | AnimalsRepositoryTest

1 test passed – 1s 721ms

AnimalContractTest (jp.fieldnotes.jjug.pact.provider) 1s 721ms
test_consumer - a request for an serval 1s 721ms

```
2017-04-24 00:56:04.839  INFO 3324 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet        : F
2017-04-24 00:56:04.875  INFO 3324 --- [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet        : F
    returns a response which
      has status code 200 (OK)
      includes headers
        "Content-Type" with value "application/json" (OK)
      has a matching body (OK)
2017-04-24 00:56:05.533  INFO 3324 --- [       Thread-4] ationConfigEmbeddedWebApplicationContext : C
2017-04-24 00:56:05.535  INFO 3324 --- [       Thread-4] o.s.j.e.a.AnnotationMBeanExporter        : Ur
2017-04-24 00:56:05.537  INFO 3324 --- [       Thread-4] j.LocalContainerEntityManagerFactoryBean : C
2017-04-24 00:56:05.537  INFO 3324 --- [       Thread-4] org.hibernate.tool.hbm2ddl.SchemaExport  : HI
2017-04-24 00:56:05.543  INFO 3324 --- [       Thread-4] org.hibernate.tool.hbm2ddl.SchemaExport  : HI

Process finished with exit code 0
```

Copyright 2017 Hiroyuki Onaka

コンシューマー駆動契約のポイント

Copyright 2017 Hiroyuki Onaka

コンシューマー駆動契約のポイント

- 検証の厳密さ
- Provider Stateの扱い
- エクスペクテーションの引き渡し

Copyright 2017 Hiroyuki Onaka

検証の厳密さ

例えば、エクスペクテーションとしてレスポンスがapplication/jsonというContent-Typeを返すことを期待した場合。

サービス(プロバイダー)がapplication/json; charset=uif-8というContent-Typeを返した場合、機能的には等価だが検証には失敗する。

Copyright 2017 Hiroyuki Onaka

Provider Stateの扱い

エクスペクテーションの検証時には、プロバーダー(サービス)のバックエンドのデータベースや、対向先のサービスのモックをセットアップする必要があります。

よって、クライアントのテストとサービスのテストが、テストデータの関係を通じて密結合することになります。

Copyright 2017 Hiroyuki Onaka

上記二つの理由により、クライアントとサービスの間のやりとりを全て検証するのではなく、サービスのクライアントの契約として必要な部分に検証を絞る必要があります。

Copyright 2017 Hiroyuki Onaka

エクスペクテーションをサービス側で検証するには、クライアントが作成したエクスペクテーションをサービス側に引き渡す必要があります。

- CIサーバーの成果物(Artifact)で引き渡す
- Pact Brokerを使用する
  - https://github.com/bethesque/pact_broker

Copyright 2017 Hiroyuki Onaka

まとめ

Copyright 2017 Hiroyuki Onaka

コンシューマー駆動契約の使いどころ

- 正直、悩ましい
- サービス側をリファクタリングした時に、クライアント側でのテストなしでクライアントとサービス側のインターフェースを検証できるというのはメリット
- だが、サービス間の依存関係がそれなりに複雑でない場合は、オーバースペックではないのか

Copyright 2017 Hiroyuki Onaka

- 「テスト自動化ピラミッド」の考え方はコンシューマー駆動契約を導入しなくても有効
- アプリケーションのレイヤーそれぞれにどのようなテストを書き、テストの網をはってリスクを潰して行くかを考えていく中で、コンシューマー駆動契約の考え方も生きてくる。

Copyright 2017 Hiroyuki Onaka

まとめ

ユニットテストとエンドツーエンドテスト双方を補完し、クライアントとサービス相互のインターフェース仕様の検証に絞った使い方が出来るかが、コンシューマー駆動契約を推進する上でのポイントになります。

Copyright 2017 Hiroyuki Onaka

情報源

- Pactのドキュメント
  - https://docs.pact.io/
- 実践 Pact:マイクロサービス時代のテストツール
  - http://techlife.cookpad.com/entry/2016/06/28/164247
- すいーとみゅーじっく vol.1 TDDってなんだ
  - https://fieldnotes.booth.pm/items/484459

Copyright 2017 Hiroyuki Onaka

「Testing Java Microservices」

- 2017年夏出版予定
  - http://www.manning.com/books/testing-java-microservicess

Copyright 2017 Hiroyuki Onaka

ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- グロースエクスパートナーズ株式会社
  アーキテクチャソリューション部
  テクニカルリード
- http://hiroyuki.fieldnotes.jp/

Copyright 2017 Hiroyuki Onaka